

DataFrames

```
In [1]: # This line imports the Numpy module
import numpy as np
```

Grouping or Storing Data

Recall...

- Lists, Tuples, Sets, Dictionaries (built-in to Python)
- Arrays (found in the NumPy library)
- DataFrames (in Pandas)

Grouping Data with DataFrames (from the Pandas library)

- Gives rows and columns of data
- Rows are "individuals" or "instances"
- Columns are attributes of those individuals

DataFrames

- A DataFrame has an **index** that refers to the rows and **column names** referring to the columns

	id	host_name	neighbourhood	latitude	longitude	room_type	price	minimum_nights	number_of_reviews	availability_365
0	2384	Rebecca	Hyde Park	41.78790			60	2	178	353
1			South Lawndale	41.85495			105		395	155
2			West Town	41.90289			60		384	321
3			Lincoln Park	41.91769	-87.63788	Entire home/apt	65		49	300
4			Hyde Park	41.79612	-87.59261	Private room	21	1	44	168
...	...	...	...	...	...	...	...	...	...	...
6392	45514091	Melanie And Ryan	Near South Side	41.85574	-87.62507	Entire home/apt	146	2	0	180
6393	45514389	Melanie And Ryan	Near South Side	41.85561	-87.62342	Entire home/apt	195	2	0	180
6394	45514632	Melanie And Ryan	Near South Side	41.85584	-87.62404	Entire home/apt	493	2	0	177
6395	45515281	Ali	West Town			Private room	53	32	0	83
6396	45515581	Kendal	Douglas			Entire home/apt	61	1	0	170

6397 rows x 10 columns

How can we make a DataFrame?

```
import pandas as pd #import a new library or module
```

From this library, we want to extract the 'DataFrame' function

`pd.DataFrame()`

```
In [2]: import pandas as pd
```

```
In [3]: small_chicago = pd.DataFrame(  
    { 1:[2384,4505,7126], #ID  
      2:['Hyde Park','South Lawndale','West Town'], #neighbourhood  
      3:[60,105,60], #price  
      4:['private room', 'entire home', 'entire home']}) #room_type  
  
small_chicago
```

```
Out[3]:
```

	1	2	3	4
0	2384	Hyde Park	60	private room
1	4505	South Lawndale	105	entire home
2	7126	West Town	60	entire home

Above: DataFrame using a Dictionary

- Creates a DataFrame along a column-axis
- Keys of dictionary become the column titles

Take a look at the code below to see how we can specify the column names and indices when creating a DataFrame.

```
In [4]: small_chicago = pd.DataFrame(  
    { 'id':[2384,4505,7126], #ID  
      'neighbourhood':['Hyde Park','South Lawndale','West Town'],  
#neighbourhood  
      'price':[60,105,60], #price  
      'room type':['private room', 'entire home', 'entire home']}, index=  
    ['a','b','c']) #room_type  
  
small_chicago
```

```
Out[4]:
```

	id	neighbourhood	price	room type
a	2384	Hyde Park	60	private room
b	4505	South Lawndale	105	entire home
c	7126	West Town	60	entire home

We can create DataFrames by defining the columns (as above) or by defining the rows (as below). We might be interested in one method over the other depending on what data we are starting with, or personal preference.

```
In [5]: # created from lists.
# You specify rows here!
small_chicago = pd.DataFrame(
    [[2384, 'Hyde Park', 60, 'private room'],
     [4505, 'South Lawndale', 105, 'entire home'],
     [7126, 'West Town', 60, 'entire home']], index=['a', 'b', 'c'],
    columns = ['id', 'neighbourhood', 'price', 'room type']) #room_type

small_chicago
```

```
Out[5]:
```

	id	neighbourhood	price	room type
a	2384	Hyde Park	60	private room
b	4505	South Lawndale	105	entire home
c	7126	West Town	60	entire home

DataFrames are built from *series*

So...

- Python has lists
- Numpy has arrays
- and Pandas has series as the *basic* way to store data

What is a series?

Pandas series

Pandas defines a basic collection data type called a "series" that is analogous to a one dimensional array in numpy.

```
In [6]: Neighborhoods = pd.Series(['Hyde Park', 'South Lawndale', 'West Town'])
print(Neighborhoods)
```

```
0      Hyde Park
1  South Lawndale
2      West Town
dtype: object
```

We can use series to make DataFrames

In fact, a DataFrame is a sequence of aligned series objects

- “Aligned” means that they share the same index

Series are a bit more flexible than arrays. For example you can specify your own index for a series, and the indexes don't have to be integers.

```
In [7]: my_Neighborhoods = pd.Series(['Hyde Park', 'South Lawndale', 'West Town'],  
index = ['a', 'b', 'c'])  
print(my_Neighborhoods)
```

```
a      Hyde Park  
b  South Lawndale  
c      West Town  
dtype: object
```

We can extract elements from a series either using the numerical index or the self written index.

```
In [8]: my_Neighborhoods[1]
```

```
/var/folders/bp/n09_8ybn2t3bm81y47gj3vg00000gn/T/ipykernel_92299/3858619131.  
py:1: FutureWarning: Series.__getitem__ treating keys as positions is deprec  
ated. In a future version, integer keys will always be treated as labels (co  
nsistent with DataFrame behavior). To access a value by position, use `ser.i  
loc[pos]`  
my_Neighborhoods[1]
```

```
Out[8]: 'South Lawndale'
```

```
In [9]: my_Neighborhoods['b']
```

```
Out[9]: 'South Lawndale'
```

Thinking of a few differences between arrays and series, we have the following. Feel free to add a few cells and experiment with series before moving on.

np.array:

- supports integer-based indexing

Series:

- supports both integer- and label-based indexing
- performs operations involving the index or location

Recall: “Indexing” is referring to an element of an iterable (series, array, list, ...) by its position within the iterable. “Slicing” is referring to a subset of elements from something like an array, list, series, or dataframe based on their indices.

If we have a large amount of data, we probably will not be creating our own DataFrame. What we will do most often is to *read in a file* that contains the data!

Data can exist in many different formats. We will focus on data stored as a csv (comma separated values). Data stored as a csv looks like this:

```
id,name,host_id,host_name,neighbourhood,group,neighbourhood,latitude,longitude
2384,"Hyde Park – Walk to UChicago, 10 min to McCormick",2613,Rebecca,,Hyde
4505,394 Great Reviews. 127 y/o House. 40 yds to train.,5775,Craig & Kathlee
7126,Tiny Studio Apartment 94 Walk Score,17928,Sarah,,West Town,41.90289,-87
9811,Barbara's Hideaway – Old Town,33004,At Home Inn,,Lincoln Park,41.91769,
10610,3 Comforts of Cooperative Living,2140,Lois,,Hyde Park,41.79612,-87.592
10945,The Biddle House (#1),33004,At Home Inn,,Lincoln Park,41.91183,-87.64,
12068,Chicago GOLD COAST 1 Bedroom Condo,40731,Dominic,,Near North Side,41.9
12140,Lincoln Park Guest House,46734,Sharon And Robert,,Lincoln Park,41.9233
22362,*** Luxury in Chicago! 2BR/ 2Ba / Parking / BBQ **,85811,Craig,,West T
24833,Private Apt 1 Block to Fullerton L Red Line – Deck,101521,Red,,Lincoln
25879,Top 2/1 Block to Fullerton L Red Line Deck & Yard,101521,Red,,Lincoln
28749,"Sanitized, Huge, Quirky Bucktown Loft + Parking",27506,Laurie,,Logan
37738,Andersonville – Perfect location!,162364,Mat And Randy,,Uptown,41.9738
39742,Central guestroom! Walk everywhere!,170758,Eric,,Near North Side,41.89
44020,2 Bed MCM 1 Block to Fullerton Red line L & Garage,101521,Red,,Lincoln
71930,"Rest, Relax and Explore",334241,Michael And Veronica,,West Town,41.89
79101,5★Cubs/Riviera/AragonTRAIN2Bdrms/Ba,272671,Two For The Road,,Uptown,41
84042,The Explorer Room,334241,Michael And Veronica,,West Town,41.89615,-87.
110705,Fab Value!/ Spa /Deck/ARAGON/RIVIERA/TRAIN/CUBS/,272671,Two For The R
```

Filepaths

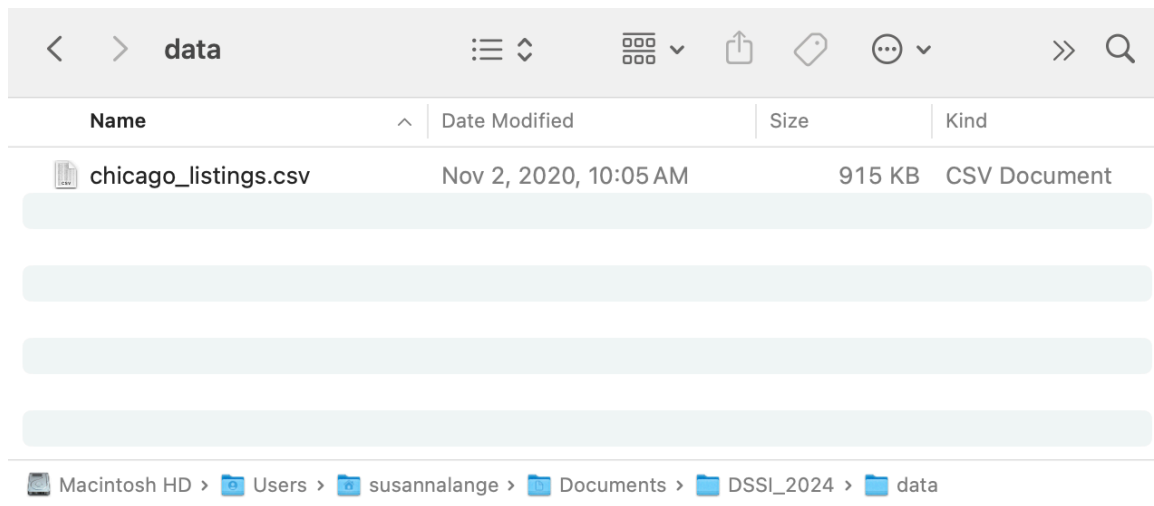
To read in data into a DataFrame, we need to tell Python where our data is located. This is done through a *filepath*. In short, filepaths tell us

- directions to access files on our local machines
- filepaths can include the names of files and directories or folders

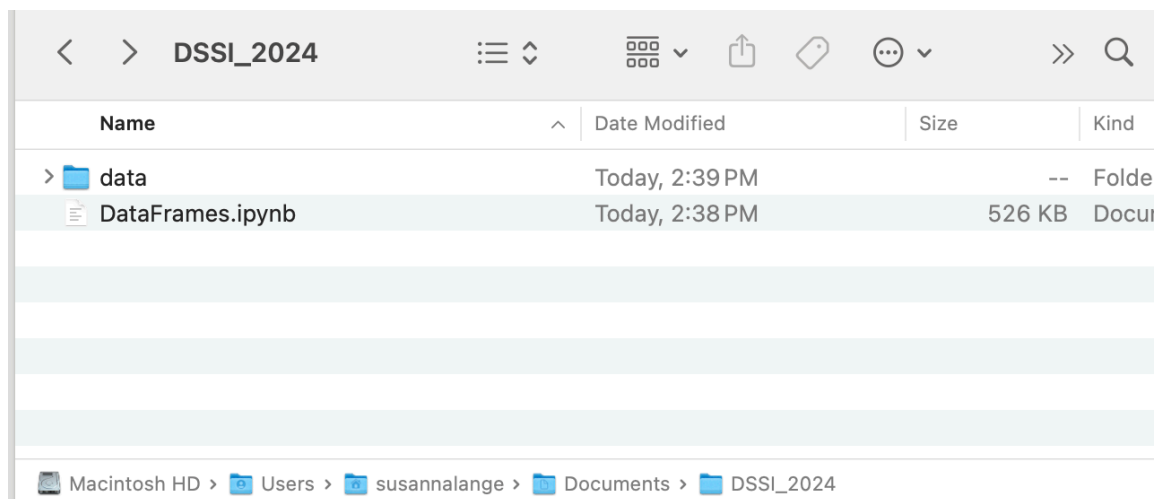
The way to access files through a filepath can be through the **absolute** filepath or the **relative** filepath.

A relative filepath contains the directions to your file *from where your document currently exists* and an absolute filepath contains directions from *anywhere*. Let's look at an example below.

Suppose we have the data we are interested in putting into a DataFrame as in the image below. Pay special attention to the string of folders on the bottom of the image. This says we are in the 'Users' folder in my directory (should be your name), and that we are in the 'data' folder which is within the DSSI folder contained in the 'Documents' folder.



The notebook I want to read in the data is in the DSSI folder.



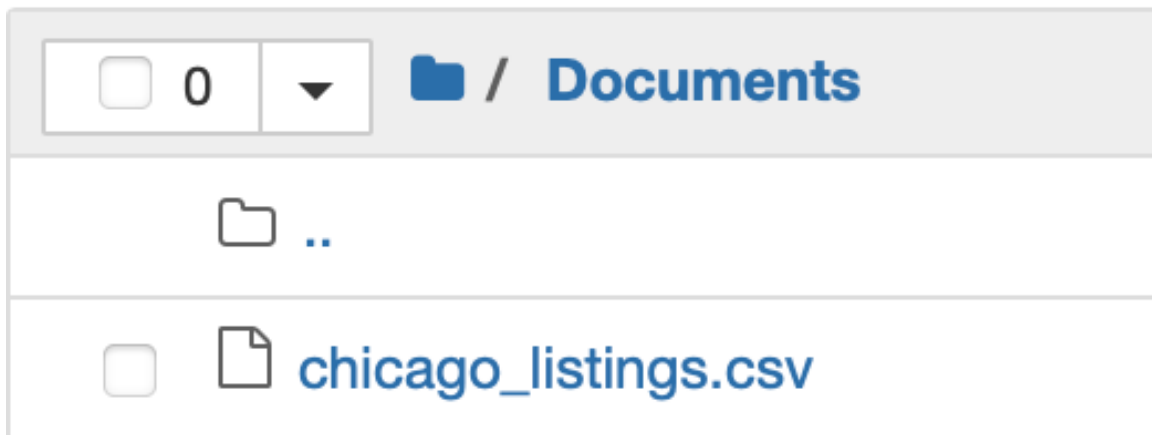
From where our notebook is located: we write the relative path

'data/chicago_listings.csv'

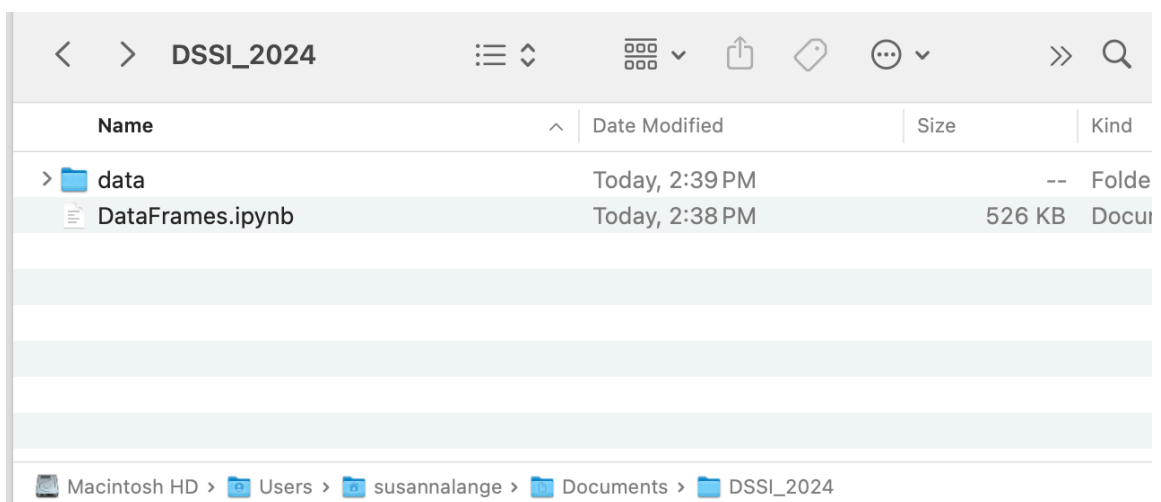
This says, I want to go into the 'data' folder and within the data folder I am looking for something called 'chicago_listings.csv'.

Another possibility

Your data might not be in the same setup as above. Suppose we have a different setup as seen below. Take a moment to determine how the below example is different than our example above.



Notebook to read in data



We should have seen that the data is stored in a different location, but the 'DataFrames.ipynb' file has stayed the same. Instead of going inside a folder to find the data, we need to go back a folder - inside the 'Documents' folder. We can specify this by a `'../'` in our filepath.

From where our notebook is located, a relative path is below

`'../chicago_listings.csv'`

Accessing files on Google Drive from Colab

We need to link our Colab notebook to our Google Drive so we can both save and load our data files. To do this we need to mount our Google Drive locally. Please be mindful of repeating this step.

```
In [85]: from google.colab import drive
drive.mount('/content/gdrive')
```

```
-----  
ModuleNotFoundError                                Traceback (most recent call last)  
Cell In[85], line 1  
----> 1 from google.colab import drive  
      2 drive.mount('/content/gdrive')  
  
ModuleNotFoundError: No module named 'google.colab'
```

Now we want to explore this larger dataset, and make sure we can read it into this notebook. Make sure your drive is mounted correctly and run the following code. Note, the line below contains an absolute filepath.

```
In [86]: chicago_airbnb_df = pd.read_csv('AirBnB_chicago_listings.csv', sep=',',  
      keep_default_na=False)  
      print(len(chicago_airbnb_df))
```

```
6397
```

Summary

- To navigate one directory up in a filepath, the notation

'../'

- To go into a folder

'/folder_name'

Note: The rest of the notebook uses the data you are reading in above and the 'small_chicago' DataFrame. If you are unable to create a filepath to read in the data, please uncomment the lines below.

```
In [88]: chicago_airbnb_df
```


Out [88]:

	id	name	host_id	host_name	neighbourhood	latitude	long
0	2384	Hyde Park - Walk to UChicago, 10 min to McCormick	2613	Rebecca	Hyde Park	41.78790	-87.6
1	4505	394 Great Reviews. 127 y/o House. 40 yds to tr...	5775	Craig & Kathleen	South Lawndale	41.85495	-87.6
2	7126	Tiny Studio Apartment 94 Walk Score	17928	Sarah	West Town	41.90289	-87.6
3	9811	Barbara's Hideaway - Old Town	33004	At Home Inn	Lincoln Park	41.91769	-87.6
4	10610	3 Comforts of Cooperative Living	2140	Lois	Hyde Park	41.79612	-87.6
...	...	...	...	...	...	...	...
6392	45514091	Best view of Chicago downtown + optional parking	367907062	Melanie And Ryan	Near South Side	41.85574	-87.6
6393	45514389	Luxurious 3Br/2Ba with optional parking and View	367907062	Melanie And Ryan	Near South Side	41.85561	-87.6
6394	45514632	Two story Penthouse w private Rooftop & Great ...	367907062	Melanie And Ryan	Near South Side	41.85584	-87.6
6395	45515281	West Town Private Room	347064639	Ali	West Town	41.89005	-87.6
6396	45515581	Chicago Themed Private Garden Apartment	19541995	Kendal	Douglas	41.82964	-87.6

6397 rows x 15 columns

More about this data: This data is from Airbnb Open Data. It contains information from Chicago in 2017-2019. Column names and what they mean are given below.

id - The id of the airbnb

name - The name of the airbnb

host_id - The id of the host

host_name - The name of the host

neighborhood - Which area the airbnb belongs to

latitude - The latitude of the position

longitude - The longitude of the position

room_type - Entire home/apt, Private room or Other

price - The price of the apartment(per day)

minimum_nights - The least number of nights needed to book

number_of_reviews - The total number of the reviews

last_review - Date of last review

reviews_per_month - Number of reviews the airbnb can receive per month

calculated_host_listings_count - The total number of listings of the host

availability_365 - The available days

Now we have a DataFrame. What can we do with it?!

Basic stuff first

Change the '2' below to a few different numbers and determine what the .head(2) is doing.

```
In [89]: chicago_airbnb_df.head(2)
```

		id	name	host_id	host_name	neighbourhood	latitude	longitude	room_
Out [89]:									
	0	2384	Hyde Park - Walk to UChicago, 10 min to McCormick	2613	Rebecca	Hyde Park	41.78790	-87.58780	P
	1	4505	394 Great Reviews. 127 y/o House. 40 yds to tr...	5775	Craig & Kathleen	South Lawndale	41.85495	-87.69696	hom

Attributes: Properties of DataFrames

In [90]: *#We could ask for the shape – dimensions – of the DataFrame*
`chicago_airbnb_df.shape`

Out[90]: (6397, 15)

In [91]: *#We could find how many rows.*
#This isn't a method or attribute though....
`len(chicago_airbnb_df)`

Out[91]: 6397

In [15]: `chicago_airbnb_df.columns`

Out[15]: Index(['id', 'name', 'host_id', 'host_name', 'neighbourhood', 'latitude',
'longitude', 'room_type', 'price', 'minimum_nights',
'number_of_reviews', 'last_review', 'reviews_per_month',
'calculated_host_listings_count', 'availability_365'],
dtype='object')

In [16]: `chicago_airbnb_df.dtypes`

Out[16]:

id	int64
name	object
host_id	int64
host_name	object
neighbourhood	object
latitude	float64
longitude	float64
room_type	object
price	int64
minimum_nights	int64
number_of_reviews	int64
last_review	object
reviews_per_month	object
calculated_host_listings_count	int64
availability_365	int64
dtype:	object

Common DataFrame Attributes

- `.shape`
- `.columns`
- `.index`
- `.dtypes`

DataFrame Extraction

```
In [17]: #We can extract an entire column!  
small_chicago['neighbourhood']
```

```
Out[17]: a      Hyde Park  
b      South Lawndale  
c      West Town  
Name: neighbourhood, dtype: object
```

```
In [18]: # There is often more than one way to do something  
small_chicago.neighbourhood
```

```
Out[18]: a      Hyde Park  
b      South Lawndale  
c      West Town  
Name: neighbourhood, dtype: object
```

```
In [19]: #Is this a series or a DataFrame?  
type(small_chicago['neighbourhood'])
```

```
Out[19]: pandas.core.series.Series
```

What if I wanted to extract as a DataFrame?! (What is the difference between the line of code below that gives a DataFrame)

```
In [20]: small_chicago[['neighbourhood']]
```

```
Out[20]:
```

	neighbourhood
a	Hyde Park
b	South Lawndale
c	West Town

```
In [21]: # Can we extract multiple columns?!...Yes!  
small_chicago[['id', 'neighbourhood']]
```

Out [21]:

	id	neighbourhood
a	2384	Hyde Park
b	4505	South Lawndale
c	7126	West Town

We can extract a single element

In [22]: `small_chicago['neighbourhood'][0]`

```
/var/folders/bp/n09_8ybn2t3bm81y47gj3vg00000gn/T/ipykernel_92299/1905971040.py:1: FutureWarning: Series.__getitem__ treating keys as positions is deprecated. In a future version, integer keys will always be treated as labels (consistent with DataFrame behavior). To access a value by position, use `ser.iloc[pos]`
small_chicago['neighbourhood'][0]
```

Out [22]: 'Hyde Park'

Above we extracted columns...What about rows?

In [23]: `#You can also choose rows by slicing`
`small_chicago[1:3] #starting at index 1 up to but excluding index 3`

Out [23]:

	id	neighbourhood	price	room type
b	4505	South Lawndale	105	entire home
c	7126	West Town	60	entire home

In [24]: `small_chicago[0:3:2]`

Out [24]:

	id	neighbourhood	price	room type
a	2384	Hyde Park	60	private room
c	7126	West Town	60	entire home

In [25]: `#combining this with column extraction`
`small_chicago[['id', 'neighbourhood']][0:3:2]`

Out [25]:

	id	neighbourhood
a	2384	Hyde Park
c	7126	West Town

What did we just say?

Basic descriptions of DataFrame:

```
dataframe.head()
```

```
dataframe.columns
```

```
dataframe.shape
```

```
dataframe.dtypes
```

```
len(dataframe)
```

We can use basic extraction to extract an entire column with either of the following:

```
dataframe.column_name
```

```
dataframe['column_name']
```

We can extract rows with

```
dataframe[start:end] # giving us rows [start, end)
```

We can extract multiple columns by:

```
dataframe[['col_name1', 'col_name2']]
```

Selecting based on label, integer index, and condition (More extraction techniques)

We can extract entire columns or entire rows, but we might want to extract smaller subsets of data. Let's see a few more extraction techniques.

We can use `.loc[]` if we want to extract via label-based selection that can subset dataframes

We can use `.iloc[]` if we want to extract via integer-based (index based) selection.

Let's take a look at these extraction techniques with the 'small_chicago' dataset so that we can print out exactly what is going on.

```
In [26]: small_chicago
```

```
Out[26]:
```

	id	neighbourhood	price	room type
a	2384	Hyde Park	60	private room
b	4505	South Lawndale	105	entire home
c	7126	West Town	60	entire home

```
In [27]: small_chicago.loc['a','id'] #row label and column label
```

```
Out[27]: 2384
```

```
In [28]: small_chicago.loc[['a', 'c'],'id']
```

```
Out[28]: a    2384  
c    7126  
Name: id, dtype: int64
```

```
In [29]: small_chicago.loc['a':'c','id':'price']
```

```
Out[29]:
```

	id	neighbourhood	price
a	2384	Hyde Park	60
b	4505	South Lawndale	105
c	7126	West Town	60

'iloc': integer-based (index based) selection method

```
In [30]: small_chicago
```

```
Out[30]:
```

	id	neighbourhood	price	room type
a	2384	Hyde Park	60	private room
b	4505	South Lawndale	105	entire home
c	7126	West Town	60	entire home

```
In [31]: small_chicago.iloc[0,0]
```

```
Out[31]: 2384
```

```
In [32]: small_chicago.iloc[0:2, 0:3]#rows first, then columns
```

```
Out[32]:
```

	id	neighbourhood	price
a	2384	Hyde Park	60
b	4505	South Lawndale	105

Extract by condition

Dataframes can also be selected based on condition using `.loc[]`

That is, based on conditions that equate to `True` Boolean values.

```
In [33]: #loc uses the above to keep only those rows that are "True"  
small_chicago.loc[small_chicago.neighbourhood == 'Hyde Park']
```

Out[33]:

	id	neighbourhood	price	room type
--	----	---------------	-------	-----------

a	2384	Hyde Park	60	private room
---	------	-----------	----	--------------

In [34]: `small_chicago.neighbourhood == 'Hyde Park' #we need double = here bc single = is for assignment`

Out[34]:

```
a    True
b    False
c    False
Name: neighbourhood, dtype: bool
```

In [35]: `#as usual, there's alternative syntax to do this`
`small_chicago['neighbourhood'] == 'Hyde Park'`

Out[35]:

```
a    True
b    False
c    False
Name: neighbourhood, dtype: bool
```

In [36]: `small_chicago.loc[small_chicago.neighbourhood == 'Hyde Park']`

Out[36]:

	id	neighbourhood	price	room type
--	----	---------------	-------	-----------

a	2384	Hyde Park	60	private room
---	------	-----------	----	--------------

Note these two options produce the same result!

You can choose which one you like best.

```
small_chicago.loc[small_chicago.neighbourhood == 'Hyde Park']
```

```
small_chicago[small_chicago.neighbourhood == 'Hyde Park']
```

You can subset on multiple conditions with the use of bitwise operators:

`&` for and

`|` for or

These perform element-wise logical 'and' 'or' operations

Suppose I am interested in...

an Airbnb in Hyde Park **or** an Airbnb less than \$80

```
(small_chicago.neighbourhood == 'Hyde Park') | (small_chicago.price < 80)
```

(It evaluates the truth value of each column of the original DataFrame)


```
In [37]: (small_chicago.neighbourhood == 'Hyde Park') | (small_chicago.price < 80)
```

```
Out[37]: a    True
b    False
c    True
dtype: bool
```

But we want to extract the rows which meet both of the conditions. Hence we denote the DataFrame `small_chicago` that we are extracting from.

```
In [38]: small_chicago.loc[(small_chicago.neighbourhood == 'Hyde Park') |
(small_chicago.price < 80)]
```

```
Out[38]:
```

	id	neighbourhood	price	room type
a	2384	Hyde Park	60	private room
c	7126	West Town	60	entire home

Suppose I am interested in...

an Airbnb in Hyde Park **and** an Airbnb less than \$80

```
In [39]: small_chicago.loc[(small_chicago.neighbourhood == 'Hyde Park') &
(small_chicago.price < 80)]
```

```
Out[39]:
```

	id	neighbourhood	price	room type
a	2384	Hyde Park	60	private room

Code Comprehension: Activity

Question 1: Let's look at all of our data. How many airbnb's are in the Lincoln Park neighbourhood that have an Entire home/apt?

```
In [40]: chicago_airbnb_df.head(3)
```

Out [40]:

	id	name	host_id	host_name	neighbourhood	latitude	longitude	room_
0	2384	Hyde Park - Walk to UChicago, 10 min to McCormick	2613	Rebecca	Hyde Park	41.78790	-87.58780	P
1	4505	394 Great Reviews. 127 y/o House. 40 yds to tr...	5775	Craig & Kathleen	South Lawndale	41.85495	-87.69696	hom
2	7126	Tiny Studio Apartment 94 Walk Score	17928	Sarah	West Town	41.90289	-87.68182	hom

In [41]:

```
lincoln_park_homes =
chicago_airbnb_df.loc[(chicago_airbnb_df.neighbourhood == 'Lincoln Park')
& (chicago_airbnb_df.room_type == 'Entire home/apt')]
len(lincoln_park_homes)
```

Out [41]: 260

► Answer here. Click to reveal.

Question 2: Of these, how many also have at least 75 reviews?

In [42]:

```
len(lincoln_park_homes.loc[lincoln_park_homes.number_of_reviews >= 75])
```

Out [42]: 61

► Answer here. Click to reveal.

Some methods help us clean the data!

What are methods again?

Methods are particular built-in functions that work on objects in Python. There are specific methods that work for all DataFrame objects!

Methods for DataFrames take the form

```
df.method()
```

Note, the '.head()' is a method, but '.shape' is an attribute or property. Methods have the () and attributes do not because we can change the argument (or what we input) into the method.

You might have noticed the UK English spelling 'neighbourhood' in the DataFrame we have been working with. We can change that using a method 'rename()' By specifying in the argument (inside the parentheses) a dictionary that maps the current column name to the desired column name.

```
In [43]: small_chicago.rename(columns = {"neighbourhood": "neighborhood"})
```

```
Out[43]:
```

	id	neighborhood	price	room type
a	2384	Hyde Park	60	private room
b	4505	South Lawndale	105	entire home
c	7126	West Town	60	entire home

Did this change the original DataFrame?!

```
In [44]: small_chicago
```

```
Out[44]:
```

	id	neighbourhood	price	room type
a	2384	Hyde Park	60	private room
b	4505	South Lawndale	105	entire home
c	7126	West Town	60	entire home

If we want to change the DataFrame, many methods (but not all) have an argument *inplace* that says we want to make a change in place or overwrite the current DataFrame. We can do this using the line below.

```
In [45]: small_chicago.rename(columns = {"neighbourhood": "neighborhood"}, inplace = True)
small_chicago
```

```
Out[45]:
```

	id	neighborhood	price	room type
a	2384	Hyde Park	60	private room
b	4505	South Lawndale	105	entire home
c	7126	West Town	60	entire home

```
In [46]: #OR this works if inplace = True is not an argument to the given method

#small_chicago = small_chicago.rename(columns =
#{ "neighbourhood": "neighborhood" })
#small_chicago
```

A note about overwriting DataFrames and being careful

Suppose we try to assign the chicago DataFrame to another variable as below with the intent to keep a copy of the original and make changes on the newly named DataFrame.

```
In [47]: small_chicago_2 = small_chicago
```

```
In [48]: small_chicago_2.iloc[0] = 3
small_chicago_2
```

```
Out[48]:
```

	id	neighborhood	price	room type
a	3	3	3	3
b	4505	South Lawndale	105	entire home
c	7126	West Town	60	entire home

```
In [49]: small_chicago
```

```
Out[49]:
```

	id	neighborhood	price	room type
a	3	3	3	3
b	4505	South Lawndale	105	entire home
c	7126	West Town	60	entire home

We saw that we also made changes to the original!!!

```
In [50]: #we need to define this again bc our original is lost!
small_chicago = pd.DataFrame(
    { 'id':[2384,4505,7126], #ID
      'neighbourhood':['Hyde Park','South Lawndale','West Town'],
#neighbourhood
      'price':[60,105,60], #price
      'room type':['private room', 'entire home', 'entire home']}, index=
    ['a','b','c']) #room_type

small_chicago
```

```
Out[50]:
```

	id	neighbourhood	price	room type
a	2384	Hyde Park	60	private room
b	4505	South Lawndale	105	entire home
c	7126	West Town	60	entire home

If we want to keep a copy of the original data unchanged, then we can first use the `.copy()` method to create a copy of the data.

```
In [51]: small_chicago_3 = small_chicago.copy()
small_chicago_3.iloc[0] = 3
small_chicago_3
```

```
Out[51]:
```

	id	neighbourhood	price	room type
a	3		3	3
b	4505	South Lawndale	105	entire home
c	7126	West Town	60	entire home

```
In [52]: small_chicago
```

```
Out[52]:
```

	id	neighbourhood	price	room type
a	2384	Hyde Park	60	private room
b	4505	South Lawndale	105	entire home
c	7126	West Town	60	entire home

Back to methods for 'cleaning'

```
In [53]: #We could drop a column (or columns) of the data
small_chicago_clean = small_chicago.copy()
small_chicago_clean = small_chicago_clean.drop(columns=['id'])
small_chicago_clean
```

```
Out[53]:
```

	neighbourhood	price	room type
a	Hyde Park	60	private room
b	South Lawndale	105	entire home
c	West Town	60	entire home

What about missing data? Or data not input correctly?

The `'.describe()'` method tells us summary information that might be useful in finding inaccurate data or even missing data.

```
In [54]: chicago_airbnb_df.describe()
```

Out [54]:

	id	host_id	latitude	longitude	price	minimum
count	6.397000e+03	6.397000e+03	6397.000000	6397.000000	6397.000000	63
mean	2.813857e+07	9.850262e+07	41.899049	-87.664042	153.020009	
std	1.288337e+07	9.990222e+07	0.058929	0.042414	376.207706	
min	2.384000e+03	2.140000e+03	41.647360	-87.846810	0.000000	
25%	1.875555e+07	1.705221e+07	41.872740	-87.687460	64.000000	
50%	2.994743e+07	5.738786e+07	41.901860	-87.660880	99.000000	
75%	3.959279e+07	1.580558e+08	41.939780	-87.633160	155.000000	
max	4.551558e+07	3.679071e+08	42.022510	-87.537520	10000.000000	5

For example, it seems odd that we have some AirBnB's that are priced over \$5000!! We can extract those data points to try to understand what is happening.

In [55]: `chicago_airbnb_df.loc[chicago_airbnb_df.price >=5000]`

Out[55]:

	id	name	host_id	host_name	neighbourhood	latitude	long
648	8053583	Single Bedroom in Wicker Park Modern Mansion	3573488	Kristina	West Town	41.90800	-87.6
1245	15287352	Wrigleyville Mansion - Sleeps 16+	45401863	Joe	Lake View	41.95112	-87.6
1707	19643452	Large 1 bedroom with Private Bath	4404730	Kyle	Lake View	41.93917	-87.6
3938	35421741	BIG Unit, Wrigleyville, Step to Wrigley, Metro	34697612	Harry	Lake View	41.95100	-87.6
4144	36667074	Steps from Mag Mile - King Bed Boutique Hotel	265201380	St Jane	Loop	41.88714	-87.6
4145	36668214	Steps from Mag Mile - Suite in Boutique Hotel	265201380	St Jane	Loop	41.88607	-87.6
4146	36668235	Steps from Mag Mile - 2 Queen Beds Boutique Hotel	265201380	St Jane	Loop	41.88621	-87.6
5558	42566731	The Robey, Urban King Accessible	317101665	The Robey Hotel	West Town	41.91142	-87.6

We see some concerning values that we might need more information to explore. For your summer project (and any future project), it's good practice to take the following actions when coming across missing data:

- Check the codebook, see if there are values set as NaN.
- Confirm with your mentor/external mentor if these values are important.
- Don't just drop them without considering the consequences!!

We can use the `.isnull()` method to check for NaN values in specific columns

```
In [56]: chicago_airbnb_df[chicago_airbnb_df['last_review'].isnull()].head()
```

```
Out[56]:
```

	id	name	host_id	host_name	neighbourhood	latitude	longitude	room_type	price
--	----	------	---------	-----------	---------------	----------	-----------	-----------	-------

Note: This is just the very start of the 'cleaning' process and in general includes making sure data is

- accurate
- complete
- consistent

Besides looking for NaN or missing data...we might have inconsistent data

What is inconsistent about the data below?

```
In [57]: #ignore this code, suppose we start with this data below!
small_chicago_messy = small_chicago_clean.copy()
small_chicago_messy.loc['c', 'room type'] = 'Entire home'
small_chicago_messy
```

```
Out[57]:
```

	neighbourhood	price	room type
a	Hyde Park	60	private room
b	South Lawndale	105	entire home
c	West Town	60	Entire home

How should we fix this??

Come up with a plan. (Note there are many ways to fix the issue above)

► Possible solution here

► Another possible solution.

Sorting rows by values

Another important method to explore the data is the `sort_values` method

```
.sort_values()
```


This method takes the *argument* of how we want to sort the data

```
In [58]: #Default sort is ascending
small_chicago_clean.sort_values(by='price')
```

```
Out [58]:
```

	neighbourhood	price	room type
a	Hyde Park	60	private room
c	West Town	60	entire home
b	South Lawndale	105	entire home

```
In [59]: #Can override that default if you like
small_chicago_clean.sort_values(by='price', ascending=False)
```

```
Out [59]:
```

	neighbourhood	price	room type
b	South Lawndale	105	entire home
a	Hyde Park	60	private room
c	West Town	60	entire home

Operations to summarize the data!

There are options to summarize the data that come from methods and also *built-in functions*. These are functions that come directly from Python, or from an imported library like NumPy. We have seen a few of these already! Some examples of built-in functions include the following:

- `print()`
- `type()`
- `len()`
- `max()`
- `np.max()` (Built in to the Numpy Library)

Run the following cells to explore.

```
In [60]: print("Print is a very useful built-in function. We don't need to import
any libraries to use print()")
```

```
Print is a very useful built-in function. We don't need to import any libraries to use print()
```

```
In [61]: len('Chicago')
```

Out [61]:

7

In [62]:

```
max(13,2)
```

Out [62]:

13

In [63]:

```
max(13,2,-5,500)
```

Out [63]:

500

In [64]:

```
#this built-in function is also in the NumPy library  
np.max([13,2,-5,500])
```

Out [64]:

500

Usually there is more than one way to approach a problem. We see below that we can take the mean using a method, or a built-in function.

In [65]:

```
#this is a method  
small_chicago_clean['price'].mean() #on a series
```

Out [65]:

75.0

In [66]:

```
#this is using a built-in NumPy function  
np.mean(small_chicago_clean[['price']]) # on a dataframe
```

Out [66]:

75.0

In [67]:

```
#this is using a built-in NumPy function  
np.mean(small_chicago_clean['price']) #series
```

Out [67]:

75.0

Here's a short list of methods that can be used to determine summative information and/or statistics:

`.min()` - determines the minimum value in a dataframe/series

`.max()` - determines the maximum value in a dataframe/series

`.mean()` - determines the mean value of a dataframe/series

`.median()` - determines the median value in a dataframe/series

`.sum()` - determines the sum of the values in a dataframe/series

`.mode()` - determines the mode of a dataframe/series

`.nlargest()` - determines the largest *n* items in a series. (Default = 5)

`.nsmallest()` - determines the smallest n items in a series. (Default = 5)

`.count()` - counts all non-null items in a dataframe/series

`.value_counts` - counts the number of times an item occurs in a dataframe/series

`.nunique()` - counts the number of unique items in a dataframe/series

`.unique()` - lists all of the unique items in a dataframe/series

`.describe()` - generates descriptive statistics of a dataframe column, including those that summarize the central tendency, dispersion and shape of a dataset's distribution, excluding null values (NaN).

A useful method that we will expand on during the first week of DSSI is the `'apply()'` method. Which can be combined with functions!!

We can apply a function to a column, or across rows of a DataFrame. Let's use the function `'np.sqrt'` as an example, which takes the square root of a number.

```
In [68]: np.sqrt(49)
```

```
Out[68]: 7.0
```

We can apply the square root function to every element of the `'price'` column of our DataFrame using the `'apply()'` method, as below.

```
In [69]: small_chicago_clean['price'].apply(np.sqrt)
```

```
Out[69]: a    7.745967
b   10.246951
c    7.745967
Name: price, dtype: float64
```

```
In [70]: #Again, this doesn't change the DataFrame
small_chicago_clean
```

```
Out[70]:
```

	neighbourhood	price	room type
a	Hyde Park	60	private room
b	South Lawndale	105	entire home
c	West Town	60	entire home

```
In [71]: #Define a new column to change the DataFrame
small_chicago_clean['sqrt
price']=small_chicago_clean['price'].apply(np.sqrt)
small_chicago_clean
```

```
Out [71]:
```

	neighbourhood	price	room type	sqrt price
a	Hyde Park	60	private room	7.745967
b	South Lawndale	105	entire home	10.246951
c	West Town	60	entire home	7.745967

```
In [72]: #we can use the .apply method on multiple columns
small_chicago_clean[['price', 'sqrt price']].apply(np.square)
```

```
Out [72]:
```

	price	sqrt price
a	3600	60.0
b	11025	105.0
c	3600	60.0

```
In [73]: small_chicago_clean
```

```
Out [73]:
```

	neighbourhood	price	room type	sqrt price
a	Hyde Park	60	private room	7.745967
b	South Lawndale	105	entire home	10.246951
c	West Town	60	entire home	7.745967

As we can see, this example is illustrative and doesn't make much sense.

As another example of using the .apply method on multiple columns, we can find the maximum of the price column and the maximum of the sqrt price column!

```
In [74]: small_chicago_clean[['price', 'sqrt price']].apply(np.max)
```

```
Out [74]: price      105.000000
sqrt price  10.246951
dtype: float64
```

We are actually using a default setting here (a setting that is automatically applied unless manually changed) - which is to apply the given function along a column. This corresponds to the 'axis' argument, and setting 'axis = 0' specifies we want to apply along a column. The below code does the same thing, with the axis argument.

```
In [75]: small_chicago_clean[['price', 'sqrt price']].apply(np.max, axis = 0)
```

```
Out [75]: price      105.000000
sqrt price  10.246951
dtype: float64
```

We can change the default if we want to apply a function across rows. This is specified by 'axis = 1', and takes the maximum across rows.

```
In [76]: small_chicago_clean[['price', 'sqrt price']].apply(max, axis = 1)
```

```
Out[76]: a    60.0  
b    105.0  
c     60.0  
dtype: float64
```

```
In [77]: #change from axis = 0 to axis =1 and make sure you understand the  
         difference.  
small_chicago_clean[['price', 'sqrt price']].apply(sum, axis = 0)
```

```
Out[77]: price      225.000000  
sqrt price    25.738884  
dtype: float64
```

We will be building on this idea when we create our own functions!

Let's summarize some of what we've done so far

Some important attributes and methods of DataFrames:

- .columns
- .shape
- .drop()
- .mean()
- sort.values(by=...)
- .apply()

We can extract slices of our DataFrame based on

- label -----> .loc
- based on index -----> .iloc
- based on condition

Note: This is an incomplete collection of a few commonly used methods. To see more [Link to Documentation: DataFrame methods/attributes](#)

Code comprehension: Activity

Let's explore the bigger chicago airbnb dataset!

```
In [78]: ## download the dataset from Canvas and load it in here
## name the dataframe chicago_airbnb
chicago_airbnb = pd.read_csv('AirBnB_chicago_listings.csv', sep=',',
keep_default_na=False)
chicago_airbnb.head()
```

```
Out[78]:
```

	id	name	host_id	host_name	neighbourhood	latitude	longitude	room
0	2384	Hyde Park - Walk to UChicago, 10 min to McCormick	2613	Rebecca	Hyde Park	41.78790	-87.58780	
1	4505	394 Great Reviews. 127 y/o House. 40 yds to tr...	5775	Craig & Kathleen	South Lawndale	41.85495	-87.69696	ho
2	7126	Tiny Studio Apartment 94 Walk Score	17928	Sarah	West Town	41.90289	-87.68182	ho
3	9811	Barbara's Hideaway - Old Town	33004	At Home Inn	Lincoln Park	41.91769	-87.63788	ho
4	10610	3 Comforts of Cooperative Living	2140	Lois	Hyde Park	41.79612	-87.59261	

Question 1: What does the below line of code do?

```
In [79]: #run this line of code
chicago_airbnb_df.tail() #is there a default if the argument is empty?
```

Out [79]:

	id	name	host_id	host_name	neighbourhood	latitude	longit
6392	45514091	Best view of Chicago downtown + optional parking	367907062	Melanie And Ryan	Near South Side	41.85574	-87.62
6393	45514389	Luxurious 3Br/2Ba with optional parking and View	367907062	Melanie And Ryan	Near South Side	41.85561	-87.62
6394	45514632	Two story Penthouse w private Rooftop & Great ...	367907062	Melanie And Ryan	Near South Side	41.85584	-87.62
6395	45515281	West Town Private Room	347064639	Ali	West Town	41.89005	-87.66
6396	45515581	Chicago Themed Private Garden Apartment	19541995	Kendal	Douglas	41.82964	-87.62

It shows the last rows of the DataFrame. With no argument it defaults to the last 5 rows.

Question 2: Print out all the columns in this dataset

In [80]: `chicago_airbnb_df.columns`

Out[80]: `Index(['id', 'name', 'host_id', 'host_name', 'neighbourhood', 'latitude', 'longitude', 'room_type', 'price', 'minimum_nights', 'number_of_reviews', 'last_review', 'reviews_per_month', 'calculated_host_listings_count', 'availability_365'], dtype='object')`

Question 3: How many neighbourhoods are included in this data?

In [81]: `chicago_airbnb_df.neighbourhood.nunique()`

Out[81]: `77`

Question 4: Find the average price of a day stay.

```
In [82]: chicago_airbnb_df.price.mean()
```

```
Out[82]: 153.02000937939658
```

Question 5: How many different room types are available? What are they?

```
In [83]: print(chicago_airbnb_df.room_type.nunique())  
chicago_airbnb_df.room_type.unique()
```

```
4
```

```
Out[83]: array(['Private room', 'Entire home/apt', 'Shared room', 'Hotel room'],  
          dtype=object)
```

Question 6: Find the maximum number of reviews of a single Airbnb.

```
In [84]: chicago_airbnb_df.number_of_reviews.max()
```

```
Out[84]: 632
```

We encourage you to spend time playing around with the DataFrames in this notebook before arriving at UChicago this summer.

Wrapping Up

During your first week of the summer program, we will build on these concepts so that you feel comfortable working with and manipulating large groups of data. See you soon!

Good work! This notebook is now complete. 😎

```
In [ ]:
```